

ServicePilot – technischer Deep-Dive

Stand: 31. August 2026

1. Zweck und Zielgruppe

Dieses Dokument richtet sich an Enterprise Architects, Cloud-Platformteams, Informationssicherheit, Datenschutz, Integrationsteams, Betrieb, Entwicklung und technische Entscheider eines potenziellen Kunden.

Es beschreibt:

- die Architektur und ihre Trust Boundaries;
- die verwendeten Google-Cloud-Komponenten;
- lokale und Cloud-Setups;
- Datenmodelle und Datenflüsse;
- Authentifizierung, Autorisierung und Secrets;
- RAG, Voice, Observability und Tests;
- im Projekt tatsächlich aufgetretene technische Probleme;
- aktuelle Grenzen der Demo und notwendige Produktionshärtung.

Wichtig: ServicePilot ist ein produktionsorientierter Demonstrator, aber noch kein vollständig integriertes Kundenproduktionssystem. Insbesondere echte Endkundenidentität, CRM/ERP, Contact Center, Telefonie, formaler CI/CD-Prozess, Disaster-Recovery-Konzept und kundenspezifische Compliance-Kontrollen müssen im Einführungsprojekt ergänzt werden.

2. Executive Technical Summary

ServicePilot kombiniert einen generativen Conversational Agent mit einem deterministischen Geschäftsbackend.



Architekturprinzip:

LLMs verstehen Sprache und organisieren den Dialog. Deterministischer Code entscheidet über Geschäftsdaten, Berechtigungen, Verfügbarkeit und Writes.

Das Sprachmodell darf keine Kunden-, Geräte-, Ticket-, Termin-, Preis- oder Transaktionsergebnisse erfinden. Ein Write-Erfolg wird erst nach dem kanonischen Backend-Ergebnis kommuniziert.

3. Verifizierter Ist-Stand der Cloud-Umgebung

3.1 Ressourcen

Komponente	Ist-Konfiguration
Projekt	servicepilot-development / 440794212320
Hauptregion	europe-west3 (Frankfurt)
Cloud Run	servicepilot-api, Gen2
Aktive Revision	servicepilot-api-00002-vwh, 100 % Traffic

Komponente	Ist-Konfiguration
Image	Artifact-Registry-Digest <code>sha256:6ea60e...c0a54a3</code>
CPU/RAM	1 vCPU / 512 MiB
Concurrency	40 Requests pro Instanz
Request Timeout	30 Sekunden
Skalierung	Minimum 0, Maximum 3 in der aktiven Revision
Cloud-Run-Ingress	<code>all</code> , aber IAM-geschützt
Runtime-Identität	<code>servicepilot-runtime@servicepilot-development.iam.gserviceaccount.com</code>
Firestore	<code>(default)</code> , Native Mode, <code>europe-west3</code> , pessimistische Concurrency
Artifact Registry	Docker Standard Repository <code>servicepilot</code> , <code>europe-west3</code>
Dialogflow Agent	<code>ServicePilot</code> , Deutsch, <code>Europe/Berlin</code> , <code>europe-west3</code>
Agent Search	<code>servicepilot-knowledge</code> , Location <code>eu</code>
Knowledge Bucket	<code>servicepilot-development-servicepilot-knowledge</code> , Location <code>EU</code>

Die Deploy-Script-Defaults erlauben bis zu 10 Instanzen; die aktive Demo-Revision wurde kostenbewusst mit maximal 3 Instanzen deployed. Für einen Kunden wird die Grenze aus Lasttest, Firestore-Verhalten, Latenzziel und Kostenbudget abgeleitet.

3.2 IAM-Ist-Zustand

Projektrolle:

- Runtime-Service-Account: `roles/datastore.user`.

Cloud-Run-Servicebindung:

- Google-managed Dialogflow Service Agent: `service-440794212320@gcp-sa-dialogflow.iam.gserviceaccount.com` mit `roles/run.invoker`;
- Demo-Benutzer mit `roles/run.invoker` für direkte technische Tests.

Der Dialogflow Service Agent besitzt außerdem seine Google-managed `roles/dialogflow.serviceAgent`-Rolle.

3.3 Agent-Ressourcen

Playbooks:

- DefaultService;
- KnowledgeSupport;
- ServiceTicket;
- AppointmentManagement;
- ComplaintManagement.

Tools:

- ServicePilotBackend;
- ServicePilotKnowledge;
- ein vorhandenes Google Built-in `code-interpreter` Tool.

Das `code-interpreter` Tool ist kein Bestandteil der ServicePilot- Geschäftsarchitektur und wird von den versionierten Playbooks nicht als Business-Tool referenziert.

Flows:

- Default Start Flow;
- AppointmentReschedule.

Voice:

- Speech Adaptation aktiv;
- STT-Modell `latest_short` für Deutsch;
- Endpointer Sensitivity 40;
- No-Speech Timeout 8 Sekunden;
- TTS Speaking Rate 0,95;
- Interaction Logging und Audioexport deaktiviert;
- strukturiertes Cloud Logging aktiviert.

4. Komponenten und technischer Hintergrund

4.1 Python, FastAPI, Pydantic und Uvicorn

Die Anwendung läuft auf Python 3.12; unterstützt wird Python 3.11 oder neuer.

- FastAPI stellt explizite REST-Endpunkte und OpenAPI-Dokumentation bereit.
- Pydantic v2 validiert alle Eingaben und Domainmodelle.
- Pydantic Settings liest `SERVICEPILOT_`-Umgebungsvariablen.
- Uvicorn ist der ASGI-Server und verarbeitet SIGTERM beim Container-Shutdown.
- Asynchrone Repository-Aufrufe vermeiden unnötiges Blockieren bei Firestore-I/O.

Das Backend enthält keine Dialogflow- oder Docker-Abhängigkeit in seiner Geschäftslogik. Der optionale Dialogflow-Python-Client ist nur Teil des Dev/Voice-Extras.

4.2 Docker und Docker Compose

Ein Dockerfile dient lokal und in Cloud Run als Build-Grundlage.

Sicherheits- und Laufzeiteigenschaften:

- Basis `python:3.12-slim`;
- nicht privilegierter Linux-Benutzer `servicepilot`;
- kein Secret im Image;
- Anwendung hört auf `0.0.0.0`;
- Cloud Run kann den injizierten `PORT` verwenden;
- ein einzelner Uvicorn-Prozess;
- stdout/stderr als Logging-Grenze.

Compose ergänzt nur Entwicklungsfunktionen:

- Port-Mapping 8000;

- Source Bind Mounts;
- Uvicorn Reload;
- Container-Healthcheck;
- `init: true` und 10 Sekunden Stop Grace Period;
- optionaler, read-only Credentials Mount für lokale Firestore-Tests.

Die Anwendung kann unabhängig von Docker mit `uvicorn app.main:app` laufen.

4.3 Cloud Run

Cloud Run betreibt das Containerimage als revisionsbasierten, autoskalierenden HTTPS-Service. Eine Revision bindet Image-Digest, Runtime-Konfiguration, Service-Account und Umgebungsvariablen unveränderlich zusammen. Änderungen erzeugen eine neue Revision; Traffic kann auf alte Revisionen zurückgerollt werden.

Das Ressourcenmodell sollte bei Architektur- und Betriebsfragen klar getrennt werden:

Ebene	Bedeutung für ServicePilot
Service	stabiler regionaler HTTPS-Endpunkt <code>servicepilot-api</code> und Traffic Policy
Revision	unveränderliche Kombination aus Image-Digest, Konfiguration, Runtime-SA, CPU/RAM und Concurrency
Instance	kurzlebige laufende Ausprägung einer Revision; verarbeitet je nach Concurrency mehrere Requests
Container	unser Python-/Uvicorn-Prozess innerhalb einer Instance

Cloud Run repliziert einen Service innerhalb seiner Region über mehrere Zonen. Das ersetzt jedoch weder eine anwendungsweite Disaster-Recovery-Strategie noch eine zweite Region. Der Entwickler verwaltet keine VMs, Nodes oder Kubernetes- Control-Plane. Google betreibt Scheduling, TLS-Endpunkt, Instanzstart und Autoscaling; ServicePilot bleibt für Container, Anwendung, Datenmodell, Autorisierung und Downstream-Kapazität verantwortlich.

Der Request-Lebenszyklus ist:

```

HTTPS + Google Front End
|
+-- Ingress Policy
+-- Cloud Run IAM / ID-Token
+-- TLS-Terminierung
v
Revision -> ausgewählte Instance -> Uvicorn/FastAPI -> Firestore
|
+-- JSON Response oder strukturierter Fehler

```

Der Container erfüllt den Cloud-Run-Vertrag: Er lauscht auf `0.0.0.0` und dem von Cloud Run injizierten `PORT` (in Cloud Run typischerweise 8080; lokal 8000), implementiert TLS nicht selbst und schreibt Logs nach stdout/stderr. Das beschreibbare Dateisystem einer Instance liegt im Speicher und ist nicht dauerhaft. Deshalb liegen Geschäftsdaten in Firestore und nicht im Container. Der konfigurierte Request Timeout umfasst auch eine gegebenenfalls notwendige Startphase; wird er überschritten, beendet Cloud Run den Request mit 504. Uvicorn verarbeitet SIGTERM für einen geordneten Shutdown, trotzdem muss jeder Write transaktional oder idempotent sein, weil verteilte Clients nach unklaren Antworten erneut senden können.

Cloud Run skaliert ohne Mindestinstanz auf null. Das reduziert Demokosten, kann aber beim ersten Request einen Cold Start verursachen. Maximum Instances begrenzt Kosten und schützt Downstream-Systeme. Google beschreibt Skalierung, private Services und den flüchtigen Container-Dateisystem-Layer in der [Cloud-Run-Übersicht](#).

Ingress und IAM sind zwei unterschiedliche Kontrollen:

- `ingress=all` macht den Endpoint netzseitig erreichbar;
- `--no-allow-unauthenticated` hält den Service IAM-privat;
- nur Principals mit `roles/run.invoker` und gültigem ID-Token dürfen aufrufen.

`ingress=all` wurde gewählt, damit der Google-managed Dialogflow Tool Caller den Cloud-Run-Endpoint erreichen kann. Für streng private Netze kann ein Service-Directory-/VPC-Design verwendet werden; das erhöht Aufbau und Kosten.

Cloud Run erwartet bei Service-to-Service-Aufrufen ein Google-signiertes ID-Token mit passender Audience. Details stehen in Googles Dokumentation zur [Service-to-Service-Authentifizierung](#).

Concurrency ist nicht mit der Anzahl der Unicorn-Worker gleichzusetzen. Eine Instance mit einem Prozess kann mehrere asynchrone Requests gleichzeitig bearbeiten. Höhere Concurrency verbessert die Auslastung, erhöht aber parallele Firestore-Aufrufe und kann bei CPU-intensiver Verarbeitung zu Latenz führen. Die richtige Einstellung muss mit realistischen Tool- und Voice-Lastprofilen gemessen werden. `max-instances` ist gleichzeitig Kostenbremse und Schutz des Downstreams, aber kein garantierter harter Ausgabenstopp.

4.4 Artifact Registry und Cloud Build

Artifact Registry speichert private, versionierte Containerimages. Das Deploy-Script erzeugt einen Zeitstempel-Tag, Cloud Build baut das Dockerfile und pusht nach:

```
eu-west3-docker.pkg.dev/<project>/servicepilot/servicepilot-api:<tag>
```

Cloud Run referenziert anschließend den unveränderlichen Digest, nicht nur den beweglichen Tag. Berechtigungen für Lesen, Schreiben und Löschen können auf Repository-Ebene getrennt werden. Google dokumentiert das private Repository und den Docker-Workflow im [Artifact-Registry-Quickstart](#).

Die Demo hat Artifact-Scanning nicht aktiviert; beim Erstellen wurde gemeldet, dass `containerscanning.googleapis.com` nicht aktiv ist. Das verhindert den Build oder das Deployment nicht. Für Produktion empfehlen wir Vulnerability Scanning, Cleanup Policies, Image-Signierung/Binary Authorization und ein definiertes Patch-SLA.

4.5 Firestore Native Mode

Firestore ist ein serverloser Dokumentenspeicher. Verwendete Collections:

- `customers`;
- `devices`;
- `tickets`;
- `appointments`;
- `appointment_slots`;
- `handovers`;
- `error_codes`.

Die Domain-ID ist gleichzeitig Dokument-ID. Kundenspezifische Listen verwenden einfache FieldFilter. Die Suche nach aktiven Duplikat-Tickets fragt zunächst den begrenzten Kundenbestand ab und filtert deterministisch weiter; dadurch ist für die Demo kein Composite Index erforderlich.

Terminverschiebung ist eine Firestore-Transaktion:

1. Termin und Zielslot werden in der Transaktion gelesen.
2. Nicht vorhandene oder bereits belegte Slots führen zu keinem Write.
3. Zielslot wird als nicht verfügbar markiert.
4. Termin erhält Slot-ID und Zeitfenster.

5. Vorheriger Slot wird wieder freigegeben.
6. Firestore committed atomar oder gar nicht.

Die Firestore Client Library wiederholt transiente Transaktionskonflikte. Google empfiehlt regionale Co-Location für geringere Kosten und Write-Latenz und weist auf Contention/Hotspots hin: [Firestore Native Best Practices](#).

Die Demo verwendet Native Mode, nicht Enterprise/MongoDB Compatibility. Das ist relevant, weil die Auswahl und der Standort einer Datenbank dauerhaft sind.

4.6 Repository- und Service-Schicht

Die Service-Schicht hängt nur von Python- `Protocol` -Interfaces ab:

- `CustomerRepository`;
- `DeviceRepository`;
- `TicketRepository`;
- `AppointmentRepository`;
- `HandoverRepository`.

Adapter:

- `InMemory`: lokale Entwicklung und deterministische Tests;
- `Firestore`: Cloud-Betrieb.

Die Auswahl erfolgt über:

```
SERVICEPILOT_PERSISTENCE_BACKEND=memory|firestore
```

Damit kann Firestore später durch PostgreSQL/Cloud SQL ersetzt werden, ohne die Business Services umzuschreiben. Transaktionssemantik muss der neue Adapter gleichwertig implementieren.

4.7 Google Conversational Agents / Dialogflow CX

Dialogflow CX verwaltet Sessions, generative Playbooks, deterministische Flows, Tools, Webhooks, STT und TTS.

4.7.1 Was ein Playbook technisch ist

Playbooks sind zielorientierte generative Agentenbausteine, keine klassisch programmierte Zustandsmaschine. Ein Playbook besteht im Wesentlichen aus:

- einem Namen und einem engen Ziel;
- natürlichsprachlichen, schrittweisen Anweisungen;
- Ein-/Ausgabeparametern;
- freigegebenen Tools und delegierbaren Playbooks/Flows;
- Beispielgesprächen mit erwarteten Actions und Ergebnissen.

Bei jedem Turn baut Dialogflow aus internen Systemvorgaben, Ziel und Anweisungen des aktiven Playbooks, passenden Few-Shot-Beispielen, den Schemas der für dieses Playbook verfügbaren Tools sowie dem aktuellen Gesprächskontext einen LLM-Prompt. Das Modell entscheidet daraus, ob es antwortet, nachfragt, ein Tool aufruft oder delegiert. Ein Wechsel zwischen Playbooks beziehungsweise zwischen Flow und Playbook übergibt zusammengefassten Kontext. Die exakte interne Promptvorlage und die Modellentscheidung sind nicht Teil unseres Anwendungscodes; Ziel, Instruktionen, Beispiele und Toolgrenzen sind dagegen versionierte Designartefakte.

Das erklärt zwei wichtige Eigenschaften:

1. Playbooks verstehen flexible Sprache, aber ihr Wortlaut und ihre Aktionswahl sind probabilistisch.
2. Eine Instruktion ist eine Verhaltensvorgabe, keine harte Transaktionsgarantie. Harte Garantien müssen in Flow, API und Backend liegen.

Google bezeichnet Beispiele ausdrücklich als Few-Shot-Beispiele. Gute Beispiele sind besonders wichtig, wenn Parameter von einem Playbook an ein anderes oder an ein Tool weitergegeben werden. ServicePilot testet deshalb nicht nur den Prompttext, sondern auch die erwartete Action Sequence. [Dialogflow CX Playbooks](#) und [Playbook Best Practices](#).

4.7.2 Aufteilung in ServicePilot

Der Default-Playbook ist der Einstiegspunkt. Kleine Task-Playbooks reduzieren Promptumfang, Verantwortungsbreite und sichtbare Tooloberfläche:

ServicePilot trennt:

Baustein	Verantwortung
DefaultService	Begrüßung, Klärung, Routing, Multi-Intent-Priorität
KnowledgeSupport	Handbuch, FAQ, Garantie, Fehlercodes
ServiceTicket	Ticketlesen und bestätigte Ticketanlage
AppointmentManagement	Terminlesen, Slots, Übergabe an Transaction Flow
ComplaintManagement	Beschwerde und menschliche Übergabe

DefaultService führt keine Backend-Operation aus. Er klassifiziert das gesamte Anliegen und delegiert mit einer Zusammenfassung. Der Spezialist erhält nur die für ihn vorgesehenen Tools. Dadurch kann beispielsweise **KnowledgeSupport** keine Terminverschiebung und **ServiceTicket** keinen RAG-Store als Geschäftsdatenquelle verwenden.

Flows sind für deterministische Zustandsmaschinen geeignet. Pages, Transition Routes, Session Parameters und Fulfillments machen den erlaubten Ablauf explizit. Deshalb liegt die kritische Terminänderung im **AppointmentReschedule** Flow und nicht im freien Playbook. Das Playbook sammelt und liest kanonische Daten; der Flow zeigt Alt- und Neutermine, verlangt eine eindeutige Bestätigung, führt genau einen Webhook-Write aus und verifiziert das kanonische Ergebnis.

4.7.3 Wie ein Playbook eine API aufruft

Für eine Ticketabfrage sieht der Laufzeitweg vereinfacht so aus:

```

Kunde: "Status von T-4711?"
|
DefaultService -> ServiceTicket
|
LLM sieht erlaubte operationIds + JSON-Schemas
|
Auswahl get_ticket, Argument ticket_id=T-4711
|
Dialogflow validiert/formt HTTP Request und erzeugt ID-Token
|
GET private Cloud Run /tickets/T-4711
|
Cloud Run IAM -> Pydantic -> Service -> Repository -> Firestore
|
200 + kanonisches Ticket ODER strukturierter Non-2xx-Fehler
|
Tool Result wird Teil des Playbook-Kontexts
|
Antwort nur auf Basis dieses Resultats

```

Das LLM ruft nicht beliebigen Python-Code auf. Es wählt eine benannte `operationId` aus der freigegebenen OpenAPI-Oberfläche und erzeugt Argumente gemäß Schema. Dialogflow führt das OpenAPI-Tool standardmäßig serverseitig aus. Bei Function Tools würde dagegen der Client den Aufruf ausführen und das Ergebnis in einem Folgeturn zurückliefern; dieses Muster verwendet ServicePilot für die Business-API nicht. Das Data-Store-Tool ist wiederum ein von Google ausgeführter Retrieval-Typ und keine FastAPI-Operation.

Schema-Validierung verhindert falsche Datentypen und unbekannte Felder, aber nicht jede fachlich falsche Kombination. Deshalb prüft das Backend weiterhin Existenz, Ownership, Zustand, Bestätigung und Idempotenz. Ein vom LLM erzeugtes `customer_id` wird nie allein dadurch vertrauenswürdig, dass es syntaktisch zum Schema passt.

4.8 OpenAPI Tool und Webhook-Authentifizierung

`conversation/servicepilot-openapi.json` beschreibt ausschließlich schmale Geschäftsoperationen. Das Deployment ersetzt die Platzhalter-Server-URL im Speicher durch die kanonische Cloud-Run-URL.

4.8.1 Wo die Contracts ins Spiel kommen

"Contract" bezeichnet hier mehrere aufeinanderliegende, maschinenprüfbare Grenzen, nicht nur ein Dokument:

Contract-Ebene	Konkretes Artefakt	Erzwingt
Conversation Contract	<code>conversation/catalog.json</code> , Golden Conversations	zuständiges Playbook, Tool-/Flow-Ownership, Bestätigungs- und Handover-Verhalten
Transport/API Contract	<code>conversation/servicepilot-openapi.json</code>	operationId, HTTP-Methode/Pfad, Parameter, JSON-Request/Response, Statuscodes
Fachlicher Contract	Pydantic-Modelle, Services, Guardrails	Ownership, Zustandsübergang, Pflichtbestätigung, Idempotenz, kanonische Entität
Fehler-Contract	<code>docs/TOOL_CONTRACTS.md</code> , Error Handler	stabiler Fehlercode, sichere Meldung, <code>retryable</code> , HTTP-Status
Persistence Contract	Repository Protocols + Adaptertests	gleiche Semantik für Memory und Firestore, atomare Terminänderung

Der OpenAPI-Contract ist für das Playbook gleichzeitig Capability-Liste und Argumentbeschreibung. Aussagekräftige `operationId`, Beschreibungen, Enums, Required-Felder und Response-Schemas helfen dem Modell, die richtige Operation zu wählen. Sie sind aber keine Autorisierungsgrenze: Cloud Run IAM autorisiert den technischen Caller, und die Service-Schicht autorisiert die fachliche Operation.

Beispiel `reschedule_appointment`:

- der Tool-Contract verlangt `appointment_id`, `customer_id`, `slot_id` und `confirmed`;
- der Conversation Contract verbietet den generativen Direktaufruf und weist die Operation ausschließlich dem `AppointmentReschedule` Flow zu;
- der Flow setzt `confirmed=true` erst nach expliziter Bestätigung;
- der Service prüft Ownership, Status und Verfügbarkeit erneut;
- die Firestore-Transaktion reserviert Zielslot und aktualisiert Termin atomar;
- Erfolg darf erst nach passender kanonischer Response ausgesprochen werden.

Damit bleibt Sicherheit erhalten, selbst wenn ein Playbook eine ungeeignete Action vorschlägt: Die nachgelagerten deterministischen Grenzen lehnen sie ab. Umgekehrt darf ein HTTP 200 mit freiem Text nicht als Erfolg gelten; der Contract erwartet die kanonische, schema-konforme Entität.

4.8.2 Authentifizierung und Versionierung

Dialogflow verwendet seinen Google-managed Service Agent, um ein ID-Token zu erzeugen. Derselbe Principal besitzt am Cloud-Run-Service `roles/run.invoker`. Es werden keine statischen API Keys, Service-Account-Keys oder Bearer Tokens im Agent gespeichert. Google beschreibt diesen Mechanismus unter [Playbook Tools](#).

Der Cloud-Run-IAM-Layer prüft Caller und Token. Das FastAPI-Backend prüft danach fachliche Regeln; es implementiert nicht erneut die Google-Tokenvalidierung.

Bei Contract-Änderungen gilt ein Expand-/Contract-Vorgehen: zuerst Backend rückwärtskompatibel erweitern, dann Tool/Playbooks umstellen, erst danach alte Felder oder Operationen entfernen. Backend und Agent werden nicht atomar als eine Ressource ausgerollt. Inkompatible Schemaänderungen brauchen daher eine neue Operation beziehungsweise API-Version und eine kontrollierte Migration.

4.9 Agent Search und hierarchisches RAG

Die Knowledge Pipeline ist dokumentenorientiert:

```

versioniertes Markdown
  |
  v
lokaler PDF Builder
  |
  v
5 PDFs + NDJSON-Metadaten -> Cloud Storage EU
  |
  v
Agent Search Data Store (eu)
  |
  +-- Layout Parser
  +-- 300 Token pro Chunk
  +-- includeAncestorHeadings=true
  |
  v
ServicePilotKnowledge Tool -> KnowledgeSupport
  
```

Der Layout Parser erkennt Titel, Überschriften, Listen und Textblöcke. Kleine Child-Chunks verbessern die Trefferpräzision; übernommene Parent-Überschriften bewahren den Abschnittskontext. Das entspricht einer hierarchischen Parent-/ Child-Idee, ohne eine eigene Vector-DB zu betreiben.

Wichtige Präzisierung: Die aktuelle ServicePilot-Implementierung ist **hierarchie-bewusstes Chunking**, aber kein vollständiger rekursiver Parent- Document-Retriever wie er mit LlamaIndex gebaut werden kann. Google zerlegt das Dokument layoutbasiert und fügt mit `includeAncestorHeadings=true` Titel und übergeordnete Überschriften in den Chunk-Kontext ein. Unsere Anwendung speichert nicht separat Child-Node, Parent-Node und deren IDs, um nach einem Child-Treffer anschließend einen frei definierten Parent-Abschnitt nachzuladen. Für die fünf kuratierten Handbücher ist die verwaltete Variante einfacher; für komplexe mehrstufige Retrieval-Algorithmen wäre LlamaIndex flexibler.

Google unterstützt beim Layout Chunking 100–500 Tokens und beschreibt `includeAncestorHeadings` explizit zur Vermeidung von Kontextverlust: [Parse and chunk documents](#).

4.9.1 Werden Embeddings gebildet?

Ja. Agent Search erzeugt standardmäßig beim Verarbeiten und Indexieren der Dokumente automatisch Vektor-Embeddings. Beim Request kombiniert Agent Search Keyword- und semantische Suche; die semantische Suche vergleicht die Bedeutung der Query mit den indexierten Repräsentationen und rankt gemeinsam mit weiteren Signalen. Es ist daher ungenau zu sagen, dass "keine Embeddings" verwendet werden.

Ebenso ungenau wäre "alle Embeddings entstehen bei jeder Antwort on the fly":

- Dokument-Embeddings werden vom Managed Service bei Ingestion/Indexierung erzeugt und gepflegt;
- die Query wird zur Suchzeit semantisch verarbeitet;
- Retrieval und Ranking laufen zur Request-Zeit;
- die Playbook-Antwort wird anschließend aus den gelieferten Treffern erzeugt.

ServicePilot wählt weder Embedding-Modell noch Dimension, betreibt keinen Embedding-Batchjob und speichert keine Vektoren selbst. Diese Details liegen hinter dem Agent-Search-Servicevertrag. Google erlaubt optional eigene Embeddings, empfiehlt für die meisten Fälle aber die automatisch erzeugten. Custom Embeddings sind derzeit eine Preview-Funktion und würden zusätzliche Metadatenfelder, Dimensionen, Schema- und Serving-Konfiguration erfordern. [Agent Search Custom Search](#) und [Custom Embeddings](#).

4.9.2 Vergleich mit LlamaIndex und eigener Vector-DB

Aspekt	Bisherige LlamaIndex-/Vector-DB-Methode	ServicePilot mit Agent Search
Loader/Parser	selbst gewählte Reader, PDF-/OCR-Pipeline	Google Layout Parser über Discovery Engine
Hierarchie	frei modellierbare Parent-/Child-Nodes, Recursive Retriever	Layout-Chunks mit angefügten Ancestor Headings
Chunking	frei programmierbar, mehrere Strategien parallel	Data-Store-weit 300 Tokens, bei Erstellung gebunden
Embeddings	Modell, Version, Dimension und Batch selbst betrieben	automatisch durch Agent Search; Modell intern verwaltet
Vector Store	z. B. Qdrant, Pinecone, Weaviate, pgvector	verwalteter Agent-Search-Index
Retrieval	top-k, Filter, Hybrid Search frei verdrahtet	verwaltete Keyword-/Semantic-Retrieval- und Ranking-Pipeline
Reranking	eigener Cross-Encoder/LLM-Reranker möglich	Google Ranking und Serving Controls
Antwort	eigener Synthesizer und Citation Formatter	Data-Store-Tool liefert Grounding für Playbook
Betrieb	Index, Backups, Skalierung, Monitoring selbst	Managed Service, weniger Infrastruktur
Transparenz	hohe Einsicht in Nodes, Scores und Modelle	weniger Kontrolle über interne Modelle und Rankingdetails

Aspekt	Bisherige LlamaIndex-/Vector-DB-Methode	ServicePilot mit Agent Search
Portabilität	höher bei eigener Abstraktionsschicht	stärkere Bindung an Discovery Engine/Dialogflow Tool

Die verwaltete Variante entfernt also nicht das RAG-Prinzip, sondern verschiebt Parsing, Embedding, Index, Hybrid Retrieval und einen Teil des Rankings in einen Google-Dienst. Die Anwendung verantwortet weiterhin Dokumentfreigabe, Versionierung, Metadaten, Suchanfrage, Quellenpflicht, Konfliktverhalten, Evaluationskorporus und Lifecycle.

LlamaIndex bleibt vorteilhaft, wenn ein Kunde ein bestimmtes Embedding-Modell, eine eigene Vector-DB, mehrstufige Parent-Erweiterung, Query Transformation, Reranker, sehr detaillierte Score-Telemetrie, On-Premises-Betrieb oder providerunabhängige Retrieval-Logik verlangt. Agent Search ist vorteilhaft, wenn geringe Betriebsverantwortung, Google-IAM, integrierte PDF-Verarbeitung, Skalierung und direkte Dialogflow-Integration wichtiger sind.

Eine spätere Rückkehr zu LlamaIndex ist architektonisch möglich: Das `ServicePilotKnowledge`-Verhalten muss als stabiler Knowledge Contract erhalten bleiben (Query hinein; Treffer mit Titel, Abschnitt, URI und Snippet heraus). Der Dialog kann dann statt des Data-Store-Tools ein schmales OpenAPI-Tool vor einem eigenen Retrieval-Service verwenden. Ein solcher Wechsel erfordert aber Neuindexierung, Retrieval-Evaluation, Betrieb und Datenschutzbewertung; er ist kein reiner Konfigurationsschalter.

4.9.3 Ingestion- und Qualitätsgrenzen

Die fünf PDFs sind abgeleitete, versionierte Demoartefakte. NDJSON liefert stabile Dokument-ID, Titel, Version, Modell und Dokumenttyp. Der Import ist inkrementell und asynchron. Parseränderungen gelten laut Google bei bestehenden Stores nur für neu importierte Dokumente; die Chunking-Konfiguration eines Stores kann nicht nachträglich geändert werden. Deshalb stoppt das Script bei abweichender Hierarchiekonfiguration, statt still einen gemischten Index zu erzeugen.

Für Produktion fehlen noch formale Freigabe- und Löschprozesse, Delta-Ingestion, Dokument-ACLs, Malware-/Prompt-Injection-Prüfung, eine Golden-Query-Suite mit Recall/Precision/Citation-Qualität und ein kontrollierter Reindex-Cutover. RAG reduziert Halluzinationen, beseitigt sie aber nicht; auch ein gefundenes Dokument kann veraltet, widersprüchlich oder böswillig formuliert sein.

Wissensregeln:

- technische Fakten nur nach Tool-Treffer;
- Titel und Abschnitt nennen;
- bei modellabhängigen Konflikten nach Modell fragen;
- leere oder widersprüchliche Treffer nicht ergänzen;
- Sicherheitsrisiken eskalieren.

4.10 Voice

Der Voice-Referenzclient nutzt den regionalen Dialogflow CX gRPC-Endpunkt:

```

16 kHz, 16-bit, Mono LINEAR16
|
v
StreamingDetectIntent
- erster Request: Session + AudioInput.config
- Folge-Requests: AudioInput.audio in 100-ms-Chunks
- Partial Transcripts
- Single Utterance / Endpointing
|
v
24 kHz LINEAR16 Response Audio

```

Barge-in übermittelt die Dauer des vorherigen Audio-Playbacks. Ein echter Telefonieadapter muss parallel aufnehmen und das Playback stoppen, sobald die Unterbrechung erkannt wird. Die Referenzimplementierung schreibt WAV-Dateien und spielt nicht selbst ab.

Google zeigt den Streaming-Vertrag unter [Detect Intent Streaming](#).

Nicht enthalten sind derzeit Carrier, SIP, PSTN, DTMF-Menüs, Contact-Center- Routing und Call Recording.

4.11 Logging und Observability

Die Anwendung schreibt einzeliliges JSON nach stdout. Cloud Run sammelt diese Ausgabe automatisch in Cloud Logging.

Request-Logfelder:

- Severity und UTC Timestamp;
- Eventname;
- Request-ID;
- HTTP-Methode;
- Route Template statt konkreter ID;
- Status;
- Protokoll;
- Latenz;
- `logging.googleapis.com/trace`, falls ein gültiger `X-Cloud-Trace-Context` vorhanden ist.

Nicht geloggt werden:

- Request Bodies;
- Query Strings;
- konkrete Kunden-, Ticket- oder Termin-IDs in der Route;
- Kundennamen.

Google erläutert strukturierte stdout-Logs und Trace-Korrelation unter [Cloud Run Logging](#).

Noch zu ergänzen sind produktionsspezifische SLOs, Alert Policies, Dashboards, Audit-Log-Retention, Security Monitoring und kundenspezifische SIEM-Anbindung.

5. API- und Datenvertrag

5.1 Endpunkte

Methode	Pfad	Funktion
GET	<code>/health</code>	Laufzeit- und Persistenzstatus
GET	<code>/customers/{customer_id}</code>	Kunde
GET	<code>/customers/{customer_id}/devices</code>	Kundengeräte
GET	<code>/customers/{customer_id}/tickets</code>	Kundentickets
GET	<code>/customers/{customer_id}/appointments</code>	Kundentermine
GET	<code>/tickets/{ticket_id}</code>	Ticket

Methode	Pfad	Funktion
POST	/tickets	validierte Ticketanlage
GET	/appointments/{appointment_id}	Termin
GET	/appointments/available-slots	freie Slots im Zeitraum
PUT	/appointments/{appointment_id}	bestätigte Verschiebung
POST	/handover	strukturierte Übergabe

5.2 Fehlervertrag

```
{
  "error": {
    "code": "APPOINTMENT_SLOT_UNAVAILABLE",
    "message": "Requested appointment slot is no longer available.",
    "retryable": false
  }
}
```

- 404: Datensatz nicht gefunden;
- 403: fachlicher Ownership-/Berechtigungskonflikt;
- 409: Business Conflict, Duplicate, Slot oder Confirmation;
- 422: Schema-/Validierungsfehler;
- 503: abstrahierter Google-/Persistenzfehler, `retryable=true`.

Providerdetails werden nicht an den Agenten oder Endkunden durchgereicht.

5.3 Idempotenz

Ticketanlage verhindert doppelte aktive Tickets für denselben Kunden, dasselbe Gerät und denselben Problemcode.

Handover:

- Client sendet stabile `handover_request_id`;
- Handover-ID wird deterministisch aus dieser Request-ID gehasht;
- identischer Retry gibt dieselbe Entität zurück;
- anderer Payload mit derselben Request-ID erzeugt `HANDOVER_REQUEST_CONFLICT`.

Termin-Write wird nach unklarer Antwort nicht blind wiederholt. Zuerst muss der kanonische Zustand erneut gelesen werden.

6. Detaillierte Datenflüsse

6.1 Ticketstatus

1. Nutzer: „Status von T-4711?“
2. DefaultService -> ServiceTicket
3. ServiceTicket -> ServicePilotBackend.get_ticket(T-4711)
4. Dialogflow erzeugt ID-Token für Cloud-Run-Audience
5. Cloud Run IAM prüft roles/run.invoker
6. FastAPI validiert Identifizier
7. TicketService -> TicketRepository.get
8. Firestore liest Dokument tickets/T-4711
9. Pydantic validiert kanonisches Dokument
10. JSON 200 an Tool
11. Agent nennt ausschließlich zurückgegebenen Status

Bei 404 endet der Pfad ohne Statusbehauptung.

6.2 Technische RAG-Frage

1. Nutzer nennt Modell/Fehler/Symptom
2. DefaultService -> KnowledgeSupport
3. KnowledgeSupport baut Query mit bekanntem Modell und Code
4. ServicePilotKnowledge -> Agent Search
5. Retrieval über Layout-Chunks plus Parent-Überschriften
6. Treffer enthält Dokumentmetadaten und Snippet
7. Agent formuliert kurze Antwort mit Titel/Abschnitt/URI
8. Kein Treffer -> explizite Wissenslücke, kein freies Auffüllen

6.3 Terminverschiebung

1. AppointmentManagement liest Kundentermine
2. AppointmentManagement liest verfügbare Slots
3. Kunde wählt genau einen Slot
4. Playbook übergibt kanonische Alt-/Neudaten an AppointmentReschedule Flow
5. Flow zeigt exakten Vorschlag
6. Dedicated Confirmation Intent muss matchen
7. Flexible Webhook führt genau einen PUT aus: confirmed=true
8. AppointmentService prüft:
 - Termin existiert
 - Termin gehört zum Kunden
 - Status ist scheduled
 - confirmed ist true
9. Firestore-Transaktion reserviert Slot und gibt alten Slot frei
10. Flow vergleicht Appointment-, Customer-, Slot-ID und Status
11. Nur bei vollständiger Übereinstimmung: succeeded

Timeout, No-Match, Abbruch oder Response-Mismatch führen nicht zu einer Erfolgsmeldung.

6.4 Handover

1. expliziter Menschenwunsch, Beschwerde, Gefahr oder zwei Fehler
2. Specialist erzeugt neutrale Summary und stabile Request-ID
3. create_handover
4. Backend validiert bekannte Customer-/Ticket-Ownership
5. Priorität:
 - complaint/technical_escalation -> high
 - human_request/repeated_failure -> normal
6. Firestore create
7. Nur queued + handover_id erlaubt Erfolgsbestätigung

Ein Menschenwunsch benötigt keine Kunden-ID. Das verhindert, dass ein schutzbedürftiger Kunde im Authentifizierungsdialog hängen bleibt. Sobald Daten eines konkreten Kunden oder Tickets verwendet werden, validiert das Backend Existenz und Ownership.

6.5 Voice

Audio wird im Client in 100-ms-Chunks gestreamt. Dialogflow liefert partielle Transkripte, finalisiert die Äußerung und verwendet danach dieselben Playbooks, Tools und Flows wie Chat. Die TTS-Antwort wird als LINEAR16 ausgegeben.

Customer IDs und Seriennummern werden vorgelesen und vor Backend-Nutzung bestätigt. Eine gesprochene Korrektur ersetzt den alten Wert. Barge-in verwirft die obsoleete Ausgabe, darf aber keine Bestätigung umgehen.

7. Authentifizierung, Autorisierung und Trust Boundaries

7.1 Identitäten

Identität	Aufgabe	Berechtigung
Deployment User/CI	Build und Deployment	getrennte Deployment-Rollen erforderlich
Cloud Run Runtime SA	Zugriff aus Backend auf Firestore	aktuell <code>roles/datastore.user</code>
Dialogflow Service Agent	Tool/Webhook-Aufruf	<code>roles/run.invoker</code> am Service
Demo User	direkter technischer Test	<code>roles/run.invoker</code> am Service
Endkunde	Gesprächsteilnehmer	derzeit keine kryptografisch gekoppelte Backend-Identität

7.2 Application Default Credentials

Im Code wird keine Credential-Datei referenziert. Die Google Client Library nutzt ADC. In Cloud Run kommt die Identität vom Metadata Server des angehängten Runtime-Service-Accounts. Lokal kann eine read-only gemountete ADC-Datei genutzt werden.

Google empfiehlt einen user-managed Service Account mit minimalen Rollen als Produktionsweg: [How ADC works](#).

In Cloud Run darf `GOOGLE_APPLICATION_CREDENTIALS` nicht auf eine Key-Datei gesetzt werden; die angehängte Service Identity ist der vorgesehene Mechanismus.

7.3 Customer Identity – wichtiger Produktionspunkt

Cloud-Run-IAM authentifiziert aktuell den technischen Caller Dialogflow, nicht den Endkunden. Der Agent verlangt explizite IDs und die Backend-Services prüfen fachliche Ownership-Beziehungen, aber die Demo bindet `C-10023` noch nicht kryptografisch an einen angemeldeten Menschen.

Für Produktion ist erforderlich:

1. Authentifizierung im Kanal, zum Beispiel Kundenportal/OIDC, Contact-Center- Identifizierung oder verifizierte Telefonie;
2. vertrauenswürdiger Identity Claim als geschützter Session Parameter;
3. Backend-Autorisierung `subject -> customer_id`;
4. Schutz davor, dass das LLM oder der Benutzer diesen Claim überschreibt;
5. Audit des Authentifizierungs- und Consent-Ereignisses;
6. Step-up Authentication für sensible Daten oder Writes.

Ohne diese Ergänzung darf die Demo nicht als vollständige Endkunden- Zugriffskontrolle bewertet werden.

7.4 Secrets

Der Kern benötigt derzeit keine Anwendungsecrets. Konfiguration ist nicht sensitiv und kommt über Umgebungsvariablen.

Wenn Integrationen Secrets benötigen, unterstützt das Deployment explizite Mappings:

```
-Secret 'INTEGRATION_API_KEY=servicepilot-integration-api-key:3'
```

Cloud Run kann Secret Manager als Environment Variable oder Volume bereitstellen. Bei Environment Variables empfiehlt Google eine feste Version statt `latest`: [Configure secrets for Cloud Run](#).

7.5 Netzwerk

Aktuell:

- Cloud Run Ingress `all`;
- keine unauthentifizierten Invoker;
- Firestore/Google APIs über Google Client Libraries;
- Knowledge Bucket mit Uniform Bucket-Level Access;
- kein VPC Connector.

Mögliche Kundenhärtung:

- Service Directory für private Tool Targets;
- VPC Service Controls;
- egress policies;
- Private Google Access;
- Cloud Armor/Load Balancer für öffentliche Kanäle;
- mTLS für externe Tools;
- getrennte Projekte für Agent, Runtime und Daten.

7.6 Datenresidenz und Datenschutz

Aktuelle Standorte:

- Cloud Run, Firestore, Artifact Registry und Agent: Frankfurt;
- Knowledge Bucket und Agent Search: EU.

Für einen Kunden müssen zusätzlich geprüft werden:

- service-spezifische Datenverarbeitung und Modellbedingungen;
- Logging- und Conversation-Retention;
- DPA/AVV;
- Subprozessoren;
- Verschlüsselungsschlüssel/CMEK-Anforderung;
- Löschkonzept und DSAR;
- Übertragung in andere Regionen bei genutzten Features;
- Voice Consent und Call Recording.

Die Demo-Handbücher und Seed-Daten sind fiktiv. Interaction Logging und Audioexport sind deaktiviert.

8. Setup und Deployment

8.1 Lokales Setup

```
Set-Location C:\Users\ralf\OneDrive\Dokumente\git\servicepilot
Copy-Item .env.example .env
docker compose build
docker compose up -d
docker compose ps
Invoke-RestMethod http://localhost:8000/health
docker compose run --rm api pytest
```

Stoppen:

```
docker compose logs --follow api
docker compose down
```

Lokale Memory-Daten werden bei Prozessneustart zurückgesetzt.

8.2 Google-Cloud-Projekt vorbereiten

```
$ProjectId = 'servicepilot-development'
$Region = 'europe-west3'
$ServiceName = 'servicepilot-api'
$Repository = 'servicepilot'
$AgentId = 'cd9cb6d6-8efa-4cdc-b3af-165a1a0508bf'
$RuntimeServiceAccount = `
  'servicepilot-runtime@servicepilot-development.iam.gserviceaccount.com'

gcloud auth login
gcloud config set project $ProjectId
gcloud services enable `
  run.googleapis.com `
  artifactregistry.googleapis.com `
  cloudbuild.googleapis.com `
  firestore.googleapis.com `
  dialogflow.googleapis.com `
  discoveryengine.googleapis.com `
  storage.googleapis.com `
  secretmanager.googleapis.com `
  --project=$ProjectId
```

Einmalig erforderlich:

- Billing-Verknüpfung;
- Firestore Native (default) in europe-west3;
- Docker Standard Repository servicepilot in europe-west3;
- Runtime-Service-Account;
- roles/datastore.user für Runtime-SA;
- Agent ServicePilot in europe-west3, Deutsch.

Keine Service-Account-Keydatei erzeugen.

8.3 Deployment-Reihenfolge

ServicePilot besteht aus mehreren Control Planes. Ein Backend-Image, eine Firestore-Datenmenge, ein Agent-Search-Index und Dialogflow-Ressourcen werden nicht in einer gemeinsamen atomaren Transaktion veröffentlicht. Die Reihenfolge ist deshalb Teil des Deployment-Contracts:

Schritt	Aktion	Warum in dieser Reihenfolge?	Verifikation/Gate
1	lokale Tests im Compose-Container	verhindert Deployment bekannter Contract-Verletzungen	gesamte pytest-Suite grün; OpenAPI und Catalog parsebar
2	Image bauen, pushen, Cloud Run deployen	erzeugt kanonische Service-URL und neue immutable Revision	Revision ready; authentifizierter /health; Image-Digest dokumentiert
3	Firestore Seed	stellt kanonische Demo-IDs für Tool- und Conversation-Tests bereit	Seed-Zähler; Stichproben-Reads; keine Produktivdaten überschreiben
4	ServicePilotBackend Tool und IAM	Tool braucht die reale URL; Dialogflow Service Agent braucht Invoker	Tool-Test für Read, Non-2xx und Auth; anonymer Call bleibt 401/403

Schritt	Aktion	Warum in dieser Reihenfolge?	Verifikation/Gate
5	Knowledge Store importieren und Tool anlegen	Playbook darf erst auf einen existierenden Store referenzieren	Import Operation erfolgreich; Golden Queries mit Quelle; Index nicht nur Upload fertig
6	AppointmentReschedule Flow/Webhook	Flow referenziert erreichbares Backend und definierte Sessionparameter	Dry Run, Flow trainiert, Decline ohne Write, Confirm mit genau einem Write
7	Spezial-Playbooks, Default, Beispiele	Referenzen auf Tools/Flow müssen bereits auflösbar sein	keine Validation Errors; Action Sequences und Delegation stichprobenartig testen
8	Voice-Profil/Referenzclient	Kanal wird erst auf einen fachlich akzeptierten Agenten geschaltet	ein Session-ID-konstanter Streaming-Test, Transkript und Output Audio
9	Live Acceptance	prüft die echten Servicegrenzen in Kombination	definierte Smoke-/Golden-Fälle, Logs, keine unerwarteten Writes

Abhängigkeitsgraph:

```

Tests
|
Cloud Run Revision -> Firestore Seed
|
+--> Backend Tool + run.invoker ----+
|                                     |
+--> Appointment Flow/Webhook -----+--> Playbooks -> Voice -> Live Acceptance
|                                     |
Knowledge PDFs -> GCS -> Index -> Knowledge Tool

```

Das Playbook-Deployment erzeugt zunächst Spezial-Playbooks, danach den Default-Router. So lassen sich `${PLAYBOOK: ...}`, `${TOOL: ...}` und `${FLOW: ...}` auf bereits vorhandene Ressourcen auflösen. Benannte Beispiele werden separat synchronisiert, weil Dialogflow wiederholte Actions bei PATCH in der Praxis angehängt hat. Die Skripte vergleichen Soll/Ist, verwenden stabile Display Names zur Auflösung und sind nach einem partiellen Fehler erneut ausführbar.

Agent Search braucht ein besonderes Gate: "Datei in Cloud Storage" und sogar "Import gestartet" bedeuten noch nicht "retrieval-ready". Das Deployment muss die asynchrone Operation abwarten und erst dann Suchfälle akzeptieren. Eine neue Chunking-Strategie bekommt eine neue Data-Store-ID; danach wird das Tool kontrolliert umgeschaltet.

Für Produktion sollte jede Stufe ein unveränderliches Release Manifest mit Git-Commit, Image-Digest, Cloud-Run-Revision, OpenAPI-Version, Agent-/Flow- Version, Knowledge-Corpus-Version und Testreport erzeugen. Zwischen Backend und Agent gilt Expand/Contract: zuerst kompatibles Backend, dann Consumer, zuletzt Entfernung alter Contract-Teile. Dialogflow Draft ist für die Demo ausreichend; Produktion benötigt Versions, Environments, Freigabe und gestuften Traffic.

Alle mutierenden PowerShell-Skripte unterstützen `-WhatIf`, soweit der jeweilige Control-Plane-Ablauf dies vorsieht. Preview vor Apply verwenden.

Die konkreten Befehle stehen im Demo-Guide unter „Vollständiger Neuaufbau“ und in `docs/CLOUD_DEPLOYMENT.md`.

8.4 Rollback

Cloud Run:

- vorherige Revision identifizieren;
- Traffic zurück auf den bekannten Digest/Revisionsnamen setzen;
- Firestore-Schema ist in der Demo rückwärtskompatibel.

Agent:

- versionierte JSON-Artefakte aus Git auschecken;
- Tools/Flow/Playbooks erneut synchronisieren;
- für echte Produktion Dialogflow Versions und Environments statt Draft Traffic verwenden.

RAG:

- Dokumentversionen und Metadaten versionieren;
- Import ist asynchron;
- Chunking-Konfiguration eines bestehenden Stores nicht stillschweigend ändern;
- bei inkompatiblem Chunking neue Data-Store-ID und kontrollierten Cutover nutzen.

9. Teststrategie

Aktuell:

154 passed, 1 warning

9.1 Warum mehrere Testebenen notwendig sind

Ein einzelner End-to-End-Test kann bei einem generativen System weder Fehler gut lokalisieren noch alle Sicherheitsinvarianten stabil prüfen. Umgekehrt beweist ein reiner Prompt-/JSON-Test nicht, dass IAM, Cloud Run, Firestore oder Agent Search funktionieren. ServicePilot teilt deshalb nach Determinismus und Servicegrenze:

Ebene	Läuft wo?	Prüft	Prüft bewusst nicht
Unit	Docker/pytest, ohne Cloud	Models, Guardrails, Services, Zustände, Idempotenz	Google IAM und echte Provider
Repository Contract	Docker/pytest, Memory/Fakes	identische Adaptersemantik, Transaktionsregeln	Firestore-Latenz und Quotas
API Integration	FastAPI via httpx	Routen, Status, JSON-Schema, Fehlerhülle, kanonische Responses	generative Toolauswahl
Conversation Contract	deterministischer Runner	Routing, Tool-/Flow-Ownership, Writes, Confirmation, Handover	exakten LLM-Wortlaut
Deployment Artifact	statische Tests	OpenAPI gegen FastAPI, Catalog-Referenzen, sichere Flags	Control-Plane-Propagation
Live Acceptance	echte GCP-Ressourcen	IAM, Cloud Run, Firestore, Agent Search, Dialogflow, Voice	vollständige Last-/Chaosabdeckung

Diese Schichtung ist zugleich die Antwort auf die Frage, wie ein probabilistisches Playbook produktionsnah getestet wird: harte Regeln werden unterhalb des LLM deterministisch getestet; die generative Ebene wird auf semantische Aktionen und Ergebnisinvarianten getestet.

9.2 API- und Contract-Tests

Die Tests vergleichen die `operationId`s und Pfade des Dialogflow-OpenAPI- Dokuments mit der realen FastAPI-Route-Tabelle. Dadurch fällt ein umbenannter Endpoint bereits vor dem Deployment auf. Positive und negative Fälle prüfen:

- Required-Felder, Pattern, Enum, Datums- und Identifier-Validierung;
- kanonische Read- und Write-Responses;

- strukturierten Non-2xx-Fehler statt HTTP 200 mit Fehlertext;
- Ownership und ungültige Zustandsübergänge;
- Idempotenz bei wiederholter Ticket-/Handover-Anfrage;
- exakt einen Write nach Bestätigung und null Writes bei Decline/Timeout;
- gleiches Serviceverhalten mit InMemory- und Firestore-Adapter.

Fakes simulieren Providerantworten reproduzierbar, ersetzen aber keinen Live- Test. Ein Firestore Fake kann beispielsweise eine atomare Codegrenze prüfen, nicht jedoch reale IAM-Fehler, Index-Propagation oder regionale Latenz.

9.3 Conversation Contracts und Golden Conversations

Der Phase-10-Korpus enthält 40 Golden Conversations in 20 Kategorien. Jede Conversation ist eine erwartete Ereignisfolge, keine starre Chatabschrift.

Golden Conversations prüfen semantisch:

- Route Sequence;
- Tool/Flow Ownership;
- Write Count;
- Confirmation Boundary;
- Canonical Success;
- Failure Outcome;
- Handover Reason.

Sie prüfen absichtlich keinen exakten generativen Wortlaut.

Beispiel einer zu prüfenden Invariante:

```
Read canonical appointment
-> list currently available slots
-> present exact old/new values
-> explicit confirmation
-> exactly one flow-owned write
-> verify returned appointment
-> only then claim success
```

Eine sprachlich elegante Antwort fällt durch, wenn sie ohne Toolquelle einen Ticketstatus erfindet. Eine anders formulierte, aber quellen- und contractkonforme Antwort besteht. Exakte Strings werden nur dort verglichen, wo der Wortlaut selbst eine Sicherheitsgrenze darstellt, etwa die vollständige Terminbestätigung.

9.4 RAG-, Flow-, Handover- und Voice-Tests

RAG-Contract-Tests prüfen lokal Corpus-Manifest, vorhandene PDFs, Dokument-IDs, 300-Token-Konfiguration und Ancestor Headings. Live Golden Queries prüfen zusätzlich, dass ein erwartetes Dokument/Abschnitt gefunden und zitiert wird. Ein guter RAG-Test misst nicht nur, ob irgendein Text kommt, sondern mindestens Quellentreffer, Modellzuordnung, Antwortbeleg und sicheres Verhalten bei leerem oder widersprüchlichem Retrieval. Für Produktion sollten Recall@k, Precision@k, Citation Correctness und Answer Faithfulness mit einem freigegebenen Query Set gemessen werden.

Der Appointment Flow wird vor und nach jedem Szenario gegen Backendzustand geprüft. Decline, mehrdeutiges "ja", belegter Slot, Timeout und abweichende Webhook-Response dürfen keine Erfolgsmeldung erzeugen. Handover-Tests prüfen stabile Request-IDs, neutrale Zusammenfassungen und den Fall, dass auch die Übergabe selbst fehlschlägt. Voice-Tests verwenden dieselben fachlichen Invarianten und ergänzen Transkription, Korrektur, Barge-in, Sessionkontinuität und Output Audio.

9.5 Failure Injection und Retry-Grenzen

Failure Injection enthält:

- Timeout;
- Connection Error;
- Backend 500;
- malformed Result;
- unbekannte IDs;
- Ownership-Verletzung;
- belegten Slot;
- Duplicate Submission;
- leeres oder widersprüchliches RAG;
- Handover Failure;
- nur eine versus zwei aufeinanderfolgende Störungen.

Jeder simulierte Fehler besitzt eine erwartete Retry- und Kommunikationsregel. Read-only Retrieval darf einmal mit sinnvoll korrigierten Parametern wiederholt werden. Ein unklar beantworteter Write wird nicht mit neuer Idempotency-ID blind wiederholt. Nach definiert wiederholten Fehlern erfolgt Handover statt einer Endlosschleife. Diese Regeln sind Teil des Contracts und nicht spontane LLM-Entscheidungen.

9.6 Live Acceptance und Release Gate

Live Acceptance verwendet frische Dialogflow-Sessions und bekannte, fiktive Seed-IDs. Vor und nach mutierenden Fällen wird der kanonische Backendzustand gelesen. Das minimale Release Gate umfasst:

1. komplette Container-Test-Suite grün;
2. neue Cloud-Run-Revision ready und authentifizierter Health-Check;
3. Backend Tool: Happy Path, Not Found und strukturierter Fehler;
4. Agent Search: mindestens eine positive, eine leere und eine modellabhängige Query;
5. Flow: Decline ohne Write und Confirm mit genau einem Write;
6. Playbook-Routing für jeden Spezialisten und Multi-Intent;
7. Handover und wiederholter Fehlerpfad;
8. Voice-Smoke-Test, falls der Release den Sprachkanal betrifft;
9. Logs auf Request-ID, Fehler und unbeabsichtigte personenbezogene Daten prüfen.

Noch nicht abgedeckte Produktionsnachweise sind Last-/Soak-Tests, Concurrency- und Quota-Grenzen, regionales Failover, Restore Drill, Penetrationstest, Dokument-Prompt-Injection, Modell-/Agent-Upgrade-Evaluation und formale SLO-/Alert-Tests. Diese Lücke ist im Produktions-Backlog bewusst ausgewiesen.

10. Betrieb, Skalierung und Kosten

10.1 Cloud Run

Request-based Billing und `min=0` halten Leerlaufkosten niedrig. Google rechnet Ressourcennutzung in 100-ms-Schritten ab und wendet einen Free Tier an; Preise sind regions- und billingabhängig: [Cloud Run Pricing](#).

Trade-offs:

Einstellung	Vorteil	Risiko/Kosten
min=0	minimale Leerlaufkosten	Cold Start
min>=1	stabile erste Latenz	Idle-Kosten
hohe Concurrency	bessere Auslastung	mehr parallele DB-/Tool-Last
niedrige max instances	Kosten-/DB-Schutz	Throttling bei Lastspitzen

Für Produktion Lasttest mit realen Gesprächsmustern durchführen. Voice und lange Toolaufrufe können andere Concurrency-Ziele erfordern als kurze REST-Reads.

10.2 Weitere Kostenquellen

- Dialogflow/Conversational Agents Requests und generative Nutzung;
- STT/TTS;
- Agent Search/Document Processing/Layout Parser;
- Firestore Reads/Writes/Storage/Backups;
- Cloud Build;
- Artifact Registry Storage und Scanning;
- Cloud Logging Ingestion/Retention;
- Cloud Storage;
- Netzwerk/VPC Connector/Load Balancer, falls ergänzt.

Billing Budgets sind standardmäßig Warnungen und kein harter Kostenstopp. Google weist ausdrücklich darauf hin: [Cloud Billing Budgets](#).

Empfehlung:

- projektweites Budget;
- zusätzliche Service-Budgets für Cloud Run, Dialogflow/Gemini, Agent Search und Firestore;
- 50/80/100-%-Schwellen;
- Forecast Alerts;
- Pub/Sub-Budgetbenachrichtigungen;
- Cloud-Run-Max-Instances;
- Artifact Cleanup Policies;
- Log-Retention;
- Kostenlabels und getrennte Umgebungsprojekte.

11. Tatsächlich aufgetretene technische Probleme und Lösungen

11.1 Billing UI verlangte genau einen Dienst

Problem: Beim Erstellen eines servicebezogenen Budgets akzeptierte die Google UI „Alle Dienste“ nicht und verlangte eine konkrete Auswahl.

Ursache: Der gewählte Budget-Scope war servicebezogen.

Lösung: Zunächst Cloud Run auswählen und für weitere kostenrelevante Dienste separate Budgets ergänzen. Wichtig: Alert Budgets stoppen Ausgaben nicht automatisch.

11.2 Firestore UI bot Enterprise/Mongo als Voreinstellung

Problem: Die Datenbankmaske zeigte Enterprise und MongoDB Compatibility als prominente Vorauswahl.

Lösung:

- Standardversion;
- Firestore Native Mode;
- Region `europa-west3`;
- regionale statt Multi-Region-Konfiguration für geringe Demokosten und Co-Location;
- tatsächlich verwendete Datenbank-ID `(default)`.

Die Auswahl von Mode, ID und Standort muss vor dem Erstellen kontrolliert werden, weil wesentliche Teile dauerhaft sind.

11.3 Artifact Registry meldete deaktiviertes Vulnerability Scanning

Problem: Repository wurde erfolgreich angelegt, aber `containerscanning.googleapis.com` war nicht aktiviert.

Lösung: Für Demo kein Blocker; Build/Push/Deploy funktionieren. Für Produktion Scanning API, Rollen, Kosten und Remediation-Prozess explizit einplanen.

11.4 Privater Cloud-Run-Service war im Browser nicht direkt erreichbar

Problem: `/docs` oder `/health` liefert ohne Token 401/403.

Ursache: `--no-allow-unauthenticated` ist absichtlich gesetzt.

Lösung:

- für CLI-Tests `gcloud auth print-identity-token`;
- für Dialogflow `roles/run.invoker` an Google-managed Dialogflow Service Agent;
- für lokale Browserdiagnose gegebenenfalls `gcloud run services proxy`;
- Service nicht für Bequemlichkeit öffentlich machen.

11.5 Ingress `all` wirkte zunächst wie „öffentlich“

Problem: Netzwerkerreichbarkeit wurde mit Autorisierung verwechselt.

Lösung: Dokumentiert, dass Ingress und IAM getrennte Ebenen sind. `all` erlaubt den Netzwerkpfad, `roles/run.invoker` plus ID-Token autorisiert den Aufruf.

11.6 Agent- und Tool-Ressourcen waren kurz nach Erstellung nicht auflösbar

Problem: Dialogflow meldete zeitweise „Referenced resource does not exist“.

Ursache: Control-Plane-Propagation zwischen Create und Referenzierung.

Lösung: begrenzte Retries mit wachsendem Delay im Deployment; idempotente Synchronisierung über Display Name und Resource Name.

11.7 Playbook Example PATCH duplizierte wiederholte Actions

Problem: Dialogflow hängt bei PATCH wiederholte `actions` an, statt sie zu ersetzen.

Lösung: Benannte Beispiele werden beim inhaltlichen Update kontrolliert gelöscht und mit geordneter Action Sequence neu erstellt. Unveränderte Beispiele bleiben unangetastet.

11.8 Dialogflow Control-Plane-Quota 429

Problem: Bei der Synchronisierung vieler Playbook-Beispiele wurde das Limit „All other requests per minute“ erreicht (`RESOURCE_EXHAUSTED`, 60/min).

Lösung:

- 429/ `RATE_LIMIT_EXCEEDED` erkennen;
- begrenztes Backoff 15/30/maximal 45 Sekunden;
- Deployment idempotent erneut ausführen;
- keine unbeschränkte Retry-Schleife.

Das betrifft Deployment/Control Plane, nicht die Business-API-Laufzeit.

11.9 Agent Search Index war nicht sofort verfügbar

Problem: PDF-Upload war beendet, Suchtreffer erschienen aber noch nicht.

Ursache: Dokumentimport und Indexierung sind asynchron.

Lösung:

- Import Operation beobachten;
- erst danach `test-knowledge-rag.ps1` ausführen;
- leeren Treffer im Agenten als Wissenslücke behandeln;
- Playbooks erst nach verfügbarem Store end-to-end akzeptieren.

11.10 Bestehendes Chunking lässt sich nicht beliebig ändern

Problem: Hierarchische Chunk-Konfiguration ist Teil des Data-Store-Vertrags.

Lösung: Deployment vergleicht 300 Tokens und `includeAncestorHeadings=true`. Bei Abweichung stoppt es und fordert eine neue Data-Store-ID statt eines stillen, möglicherweise inkonsistenten Updates.

11.11 Voice Python SDK hatte einen geänderten Message-Vertrag

Problem: Ein älteres Muster setzte `InputAudioConfig` direkt in `QueryInput.audio`. Die installierte Bibliothek erwartete `AudioInput`.

Lösung:

- Protobuf Descriptors der installierten SDK-Version inspiziert;
- erster Request: `AudioInput(config=input_config);`
- Folgequests: `AudioInput(audio=chunk);`
- ausführbarer Contract Test hinzugefügt.

11.12 User-OAuth-Token hatte kein Quota Project

Problem: Live Voice Streaming endete mit 403 und Hinweis auf fehlendes Quota Project.

Ursache: Kurzlebiges User Access Token ohne zugeordnetes Consumer Project.

Lösung: `google.oauth2.credentials.Credentials` erhält explizit `quota_project_id=servicepilot-development`. Token bleibt kurzlebig und wird nicht gespeichert.

11.13 Cold Start in kostenoptimierter Demo

Problem: Erster Request konnte merklich langsamer sein.

Ursache: Cloud Run Minimum Instances 0.

Lösung: Health-Request kurz vor der Demo. Für Produktion anhand Latenz-SLO entscheiden, ob Minimum Instances > 0 wirtschaftlich gerechtfertigt ist.

11.14 Generativer Wortlaut ist nicht stabil

Problem: Exakte Stringtests wären trotz korrektem Verhalten flaky.

Lösung: 40 Golden Conversations prüfen semantische Ergebnisse, Toolauswahl, Writes, Confirmation, Failure und Handover. Exakter Wortlaut wird nur dort geprüft, wo er Teil einer deterministischen Bestätigung ist.

12. Aktuelle Grenzen und Produktions-Backlog

Bereich	Demo-Stand	Produktionsanforderung
Endkundenidentität	IDs im Gespräch, fachliche Ownership	OIDC/Portal/CCaaS Identity, geschützter Claim, Step-up Auth
Handover	Firestore Queue Record	CRM/CCaaS Ticket, Routing, SLA, Agent Desktop
Voice	Streaming WAV Reference	SIP/PSTN/CCaaS, DTMF, Consent, Call Lifecycle
Agent Lifecycle	Draft + Scripts	Version/Environment Promotion, approvals, rollback
CI/CD	lokale PowerShell-Skripte	Pipeline, Workload Identity Federation, Artefakt-Attestierung
Infrastructure as Code	teilweise imperative gcloud/REST	Terraform/Pulumi, Policy as Code
Security	private Cloud Run, least-privilege Kern	threat model, VPC-SC, scanning, Binary Authorization, SIEM
DR	Firestore managed durability	RTO/RPO, PITR/Backup, Restore Drill
Observability	JSON logs, request/trace correlation	Metriken, SLOs, Alerting, Tracing, Runbooks
Daten	fiktive Demo-Sätze	Mapping, Migration, Data Quality, Retention, DLP
RAG	fünf kuratierte PDFs	Dokumentfreigabe, Delta Import, Evaluation, ACLs, Löschung
Last	funktionale Tests	Last-, Soak-, Chaos- und Quota-Tests

13. Typische Fragen einer Kunden-IT

Kann das LLM direkt in Firestore schreiben?

Nein. Das LLM kann nur versionierte Tooloperationen beziehungsweise den deterministischen Appointment Flow auslösen. Backend-Services und Repository- Adapter prüfen die Geschäftsregeln. Firestore-Zugang besitzt nur die Cloud-Run- Runtime-Identität.

Warum Playbooks und Flows?

Playbooks eignen sich für Sprache, Kontext und flexible Dialoge. Flows eignen sich für kontrollierte Zustände und eindeutige Transaktionsgrenzen. Die Terminänderung benötigt beide: generative Erfassung, deterministische Bestätigung und Write.

Wie verhindert ihr Halluzinationen?

Nicht durch einen einzigen Prompt, sondern durch mehrere Grenzen:

- kleine Tooloberflächen;
- kanonische Reads;
- RAG-Quellenpflicht;
- strukturierte Non-2xx-Fehler;
- keine Erfolgsmeldung ohne kanonische Entität;
- bestätigungspflichtige Writes;
- deterministischer Flow;
- Golden Regression Tests;
- Handover bei Unsicherheit.

Ist Cloud Run öffentlich?

Der Endpoint ist wegen `ingress=all` netzseitig erreichbar, aber IAM-privat. Unauthentifizierte Requests werden vor dem Container abgewiesen. Nur explizite Invoker erhalten Zugriff.

Werden Service-Account-Keys verwendet?

Nein. Cloud Run verwendet eine angehängte Runtime Service Identity über ADC; Dialogflow verwendet seinen Google-managed Service Agent. Lokal werden Credentials nur bei Bedarf read-only in Compose gemountet.

Ist der Endkunde bereits sicher authentifiziert?

Nein, nicht vollständig. Die technische Service-to-Service-Authentifizierung ist implementiert. Eine echte, kryptografisch an `customer_id` gebundene Endkundenidentität ist ein expliziter Produktionsbaustein.

Kann Firestore durch PostgreSQL ersetzt werden?

Ja. Services hängen von Repository Protocols ab. Ein PostgreSQL-Adapter muss dieselben kanonischen Reads, Idempotenz- und Transaktionsgarantien implementieren.

Wie werden Updates ausgerollt?

Backend als neues Image und Cloud-Run-Revision; Conversational-Konfiguration aus versionierten JSON-Artefakten über idempotente Skripte. Für Produktion werden CI/CD und Agent Environments empfohlen.

Wie werden Kosten begrenzt?

Scale-to-zero, Maximum Instances, regionale Co-Location, kleine Images, servicebezogene Budgets, Artifact Cleanup, Log Retention und Quota Monitoring. Budgets sind Warnungen und allein kein harter Cap.

Wie sieht Mandantenfähigkeit aus?

Die Demo ist ein einzelner fiktiver Mandant. Für Multi-Tenancy müssten Tenant ID, Identity Mapping, Datenpartitionierung, IAM, RAG-ACLs, Quotas, Keys, Logs und Supportprozesse end-to-end tenant-aware werden. Nur ein Feld im Prompt reicht nicht.

Wie wird ein Datenschutzvorfall untersucht?

Über Cloud Audit Logs, Cloud Run Request Logs, korrelierte strukturierte Application Logs, Deployment-Historie, Agent-Version und Backend-Datensatz. Für Produktion müssen Retention, SIEM, Zugriff auf Logs und Incident Runbooks formal definiert sein.

Erzeugt ServicePilot Embeddings oder passiert das bei jeder Anfrage?

Embeddings werden verwendet, aber in der aktuellen Architektur nicht von ServicePilot-Code erzeugt. Agent Search erstellt und pflegt Dokument-Embeddings bei Ingestion/Indexierung automatisch. Zur Suchzeit verarbeitet der Dienst die Query semantisch und kombiniert semantische und Keyword-Signale. Das verwendete Managed Embedding-Modell und seine Dimension sind kein von uns gepinntes Anwendungsartefakt. Eigene Embeddings sind möglich, derzeit aber nicht Teil der Implementierung.

Ist das wirklich hierarchisches RAG?

Es ist layout- und hierarchiebewusst: Chunks folgen Dokumentelementen und erhalten ihre Ancestor Headings. Es ist nicht derselbe Algorithmus wie ein frei programmierter LlamaIndex Recursive Retriever, der Child- und Parent-Nodes separat speichert und nachlädt. Wenn ein Kunde genau diese mehrstufige Semantik benötigt, kann hinter einem stabilen Knowledge-Tool-Contract wieder ein eigener LlamaIndex-Service eingesetzt werden.

Warum Agent Search statt LlamaIndex und eigener Vector-DB?

Für diese Demo reduziert Agent Search Betriebsaufwand und integriert Parsing, automatische Embeddings, Hybrid Retrieval, Ranking, Skalierung, IAM und Dialogflow-Tooling. Der Preis dafür ist weniger Kontrolle über Embeddingmodell, Scores und Retrievalpipeline sowie mehr Providerbindung. Die Entscheidung muss bei einem Kunden anhand von Kontrolle, Compliance, Betriebsteam, Datenvolumen, Retrievalqualität und Portabilität neu bewertet werden.

Wie werden neue oder geänderte Handbücher ausgerollt?

Quelle und Metadaten werden versioniert, das PDF neu erzeugt, nach Cloud Storage geladen und inkrementell importiert. Erst nach abgeschlossener asynchroner Indexierung laufen Golden Queries. Parseränderungen greifen nicht automatisch auf alte Dokumente; inkompatibles Chunking erfordert einen neuen Store und kontrollierten Tool-Cutover.

Wie werden gelöschte oder gesperrte Dokumente sicher entfernt?

Die Demo besitzt noch keinen vollständigen Produktionsprozess dafür. Benötigt werden eindeutige Dokument-IDs, Reconciliation-/Delete-Prozess, Prüfung des Indexzustands, Cache-/Antworttests, Audit Trail und gegebenenfalls ACLs pro Benutzer oder Mandant. Eine Datei nur aus dem Quell-Bucket zu löschen ist kein ausreichender Nachweis, dass sie nicht mehr im Suchindex liegt.

Kann ein manipuliertes Handbuch den Agenten steuern?

RAG-Inhalte sind Daten und müssen als untrusted input behandelt werden. ServicePilot begrenzt das Tool auf freigegebene Quellen und erlaubt technische Fakten nur mit Zitat, hat aber noch keine vollständige Content-Security-Pipeline. Produktion benötigt Publisher-Freigabe, Malware- und Prompt-Injection-Prüfung, Dokumentsignatur/Provenance, ACLs und Tests mit adversarial documents.

Garantiert OpenAPI, dass das LLM fachlich richtige Werte sendet?

Nein. OpenAPI begrenzt Operationen, Felder und Datentypen. Ein syntaktisch gültiger, aber fremder `customer_id` bleibt untrusted. Pydantic, Services, Ownership-Prüfungen, Zustandsmaschine und Transaktionen müssen jede fachliche Entscheidung erneut deterministisch prüfen.

Was passiert bei einer inkompatiblen API-Änderung?

Backend und Dialogflow werden nicht atomar deployt. Deshalb wird zuerst eine rückwärtskompatible Backend-Erweiterung ausgerollt, danach OpenAPI/Agent und erst später die alte Variante entfernt. Breaking Changes erhalten eine neue Operation oder API-Version. Contract- und Live-Tests sind das Promotion Gate.

Kann Dialogflow ein Tool versehentlich doppelt aufrufen?

Wiederholungen sind in verteilten Systemen nie vollständig auszuschließen. Read-Operationen sind wiederholbar. Kritische Writes verwenden Bestätigung, Idempotenzkennungen beziehungsweise transaktionale Zustandsprüfungen. Nach einer unklaren Antwort wird der kanonische Zustand gelesen, statt Erfolg zu erfinden oder mit neuer ID blind erneut zu schreiben.

Warum Cloud Run und nicht GKE?

Cloud Run passt zum request-getriebenen, containerisierten Backend mit kleinem Betriebsteam, Scale-to-zero und standardisiertem HTTPS/IAM. GKE wäre sinnvoll, wenn der Kunde volle Kubernetes-Kontrolle, spezielle Daemons, komplexe Netzwerk-/Sidecar-Topologien oder Workloads außerhalb des Cloud-Run-Vertrags benötigt. Diese Demo benötigt diese Komplexität nicht.

Ist eine Cloud-Run-Instance ein dauerhafter Server?

Nein. Instances sind kurzlebig, können parallel entstehen und bis auf null verschwinden. Lokaler Speicher und In-Memory-Zustand sind nicht kanonisch. Persistenz liegt in Firestore; Request-Verarbeitung und Writes müssen concurrency- und retry-sicher sein.

Was geschieht bei Modell- oder Plattformupdates von Dialogflow?

Playbooks verwenden einen Managed Generative Runtime Stack. Verhalten kann sich trotz unveränderter Instruktionen verschieben. Deshalb werden Catalog, Beispiele, Golden Conversations und Live Acceptance als Release-Evidence behandelt. Produktion benötigt zusätzlich Agent Versions/Environments, ein Upgrade-Fenster und Regressionsevaluation vor Promotion.

Welche Daten verlassen welche Trust Boundary?

Dialogflow erhält Gesprächskontext und Tool-Schemas. Der Backend-Tool-Request enthält nur die für die Operation erforderlichen IDs/Felder. Cloud Run greift mit seiner Runtime-SA auf Firestore zu. Agent Search verarbeitet freigegebene Dokumente und Queries. Vor Produktion müssen Datenklassifikation, DPA, Region/Residency, Retention, Logging, Subprozessoren und kanalspezifische Einwilligung mit dem Kunden formal bestätigt werden.

Wie wird die Retrievalqualität gegenüber der LlamaIndex-Lösung verglichen?

Beide Varianten müssen dasselbe freigegebene Query Set mit erwarteten Dokumenten, Abschnitten und zulässigen Antworten durchlaufen. Verglichen werden mindestens Recall@k, Precision@k, Citation Correctness, Faithfulness, Latenz, Kosten und sichere No-Answer-Rate. Einzelne beeindruckende Demoantworten sind kein belastbarer Benchmark.

14. Offizielle Google-Cloud-Referenzen

- [Cloud Run Overview](#)
- [Cloud Run Resource Model](#)
- [Cloud Run Container Runtime Contract](#)

- [Cloud Run Configuration](#)
- [Cloud Run Service-to-Service Authentication](#)
- [Cloud Run Logging](#)
- [Cloud Run Pricing](#)
- [Artifact Registry Docker Quickstart](#)
- [Application Default Credentials](#)
- [Cloud Run Secrets](#)
- [Firestore Native Best Practices](#)
- [Dialogflow CX Playbooks](#)
- [Dialogflow Playbook Instructions](#)
- [Dialogflow Playbook Best Practices](#)
- [Dialogflow Playbook Tools](#)
- [Dialogflow Data Store Tools](#)
- [Agent Search Custom Search Architecture](#)
- [Agent Search Parsing and Chunking](#)
- [Agent Search Custom Embeddings](#)
- [Dialogflow Streaming Detect Intent](#)
- [Cloud Billing Budgets](#)

15. Repository-Referenzen

- [README.md](#) – Entwickler-Quickstart;
- [docs/ARCHITECTURE.md](#) – kompakte Architektur;
- [docs/CLOUD_DEPLOYMENT.md](#) – Cloud Deployment;
- [docs/CONVERSATIONAL_AGENT.md](#) – Agent, Tools und Flow;
- [docs/VOICE.md](#) – Voice-Vertrag;
- [docs/TEST_STRATEGY.md](#) – Teststrategie;
- [docs/TOOL_CONTRACTS.md](#) – API-/Toolverträge;
- [conversation/catalog.json](#) – Playbooks und Beispiele;
- [conversation/appointment-reschedule-flow.json](#) – Transaction Flow;
- [conversation/servicepilot-openapi.json](#) – Agent Tool API;
- [conversation/golden_conversations.json](#) – Regression Corpus;
- [data/knowledge](#) – versionierte Knowledge Sources;
- [data/seed/demo.json](#) – fiktiver Demo-Datensatz.